Quantitative Analyses of the Performance and Efficiency Differences between Popular Instruction Set Architectures

***Abstract*—**

I. Introduction

An Instruction Set Architecture (ISA) defines how Central Processing Units (CPUs) interact with software through "instructions," the lowest level steps that programs execute. Some instructions are simple and ubiquitous, such as an ADD (add two registers) or LOAD (load data) instruction, but ISAs can include more complex instructions that aren't shared across ISAs. These complex instructions are more frequently found in "Complex" Instruction Sets, and are used significantly less than the "simple" instructions. Complex Instruction Set Computer (CISC) architectures, such as x86, and Motorola 68k, are characterized by hundreds or thousands of base instructions, with many of those instructions taking longer than the traditional one-cycle instruction, and having many different memory operations. CISC ISAs were dominant in the "early" days of computing where efficient memory usage and small code sizes were paramount. The main CISC ISA still dominant today is x86_64–the 64-bit "modern" version of the x86 ISA–and is primarily used in high-performance environments such as data centers and high-end desktop computers. Unlike the CISC ISAs pioneered by x86, Reduced Instruction Set Computer (RISC) ISAs were hitting the market in the 1980s and 1990s, characterized by fixed-width instructions, a low instruction count, and larger binary (executable file) sizes due to the reduced efficiency compression with fixed-width instructions. Over time, RISC ISAs such as ARM, PowerPC, RISC-V, and MIPS have been focused in the embedded (low power, low cost) market, with ARM having had the greatest reach in the low-power (embedded) and mobile (smartphone + laptop) markets [1].

Ever since the invention of ISAs, debates between which ISAs give "better" performance have raged on. Some say that the ISA has a negligible impact on the performance of the CPU and that the microarchitecture (the low-level implementation of the CPU itself) has a larger impact on performance and efficiency. Others say that the features of some ISAs can affect microcode efficiency, processor area, and performance. This debate is easily categorized into the debate between CISC and RISC ISAs. CISC ISAs still dominate the high-performance computing market with x86, although RISC ISAs (ARM and RISC-V) are gaining traction, even if at a slow rate. The debate between whether CISC ISAs are slower/faster than RISC ISAs, or are more/less efficient, is important to supercomputer and datacenter builders, who value raw performance and low operating costs. This research study will examine if and how ISA differences can result in real-world performance.

II. Literature Review

The earliest computers were tailored to very specific applications, resulting in programs that were not portable. This meant that a program written for one computer was not easily able to be shared from one lab to another, and such a transfer would necessitate an almost entire rewrite if the computers used did not match exactly. Through IBM's System/360 family of computers, the concept of an "architecture" became separate from the "implementation" [2]. This was the first (true) system that featured multiple implementations with different performance specs, but compatible architectures, meaning programs could be run across the entire family without having to write for one specific system. This resulted in a few innovations, namely a "storage approach", which allowed for more capacities and hierarchies of speed of memory; A new approach for Input/Output (have processors do IO, not the CPU); better data exchange because of specified data formats; and the concept of a device "family", where there were multiple "tiers" of product, so everyone from the small business to large scientific institutions could use these products [2].

In the 1980s, David Patterson and David Ditzel analyzed existing CISC architecture to find instruction frequency distributions, compiler behavior, and microarchitecture complexity. They found that in IBM 360 compilers, only 10 instructions accounted for 80% of executed instructions, 16 for 90%, 21 for 95%, and 30 for 99% [3]. They argued that CISC's complex instructions increased cycle time, complicated pipelining, and consumed silicon area that could be better used for registers and cache. They proposed a simplified instruction set with uniform instruction formats, a load-store architecture (having explicit instructions for storing and loading data, simplifying execution), and large register files optimized for compiler use. They used the IBM 801 minicomputer as an example of a great RISC machine, one of the first times that the idea of a reduced instruction set alone could result in better performance [3]. Many companies did research into the performance and efficiency of their processors, notably Intel found that as their processors got more complex, the performance would increase but energy efficiency fell sharply [4]. In Intel's testing, the Pentium 4 (Cedarmill) processor was 8x faster than the i486 processor, but consumed 38x the power [4].

The vast majority of researchers–including the Intel researchers doing the Pentium study–use the SPEC CPU benchmarks, a benchmark suite designed to compare CPU

performance across different ISAs fairly and in a way that is reproducible. Compute-intensive workloads are divided into integer (SPECint) and floating-point (SPECfp) categories. The benchmarks measure both speed (single-task performance) and rate (throughput with multiple cores/copies). SPEC defines specific compilation flags, memory requirements, and reporting requirements to ensure reproducible results [5]. At the time of its introduction in 1989, the SPEC CPU benchmarks were meant to run on real hardware, requiring millions of dollars upfront and a lot of time to fabricate the chips themselves. Nowadays, most researchers do some kind of simulation, typically with the gem5 simulator [6]. Gem5 allows researchers to model hardware at the cycle level, with enough fidelity to boot Linux-based operating systems and run full applications for multiple architectures [6]. Some researchers, when estimating the final size of their chip design, use McPAT, a Power Area and Timing framework for power draw and area analysis of different implementations of different ISAs. It supports processor configurations for under 22nm for manycore and multicore, including models for Out-of-Order processor cores, shared caches, and multiple-domain clocking. These configurations are incredibly useful when comparing the performance, cost, and efficiency of different CPU implementations [7].

Much lab research has been done to evaluate the performance and efficiency differences between ISAs using either simulations or real-world processors, though they typically target a specific field such as High-Performance Computing or low-energy environments. A study comparing the differences between many different ISA extensions found that SIMD speedups over compiled C code ranged from a 2x to 5x improvement on average, with some kernels achieving much higher speedups (up to 36. 6x for the DCT kernel on Motorola G4). Motorola's 128-bit AltiVec extension (PowerPC) performed best overall (23. 7% better than average), followed by AMD Athlon (19. 1% better than average) and Intel Pentium III (10. 9% better than average) [8]. Another study done on the differences between x86_64, ARM, and Alpha found that, while ISAs did have a measurable impact on performance, there was no significant performance difference for In-Order cores, and there was a reduced performance difference for Out-of-Order cores. This study emphasized that microarchitectural differences were more significant than just ISA changes [9].

In an analysis of ARM and RISC-V SoCs however, RISC-V CPUs were found to be 1/4th as energy-hungry as ARM CPUs, but were significantly slower (6x) than the ARM CPUs [10]. This could in part be explained by a 2019 study testing many different CPUs with many different ISAs that found that microarchitecture had a vastly greater impact on performance and efficiency than ISA alone [11]. This gives credibility to the idea that ISAs don't inherently give CPUs their performance, it's the industry requirements that give CPUs their performance. For example, a CPU design company aiming for mass-produced and cheap processors might opt for the most energy-efficient design, so CPUs are naturally going to be more energy-efficiency oriented regardless of ISA. Vector Instruction Extensions are another argument however, as when they are tested in limited thermal situations, the differences between the extensions follow a clear decrease in energy efficiency and clock speed for every new extension generation. For x86_64 specifically, many different CPU benchmarks were run on SSE, AVX, AVX2, and AVX512 vector extensions, finding that while AVX512 had the best performance in ideal scenarios, thermal limitations caused the CPU to perform worse due to decreased clock speeds and energy efficiency ratings [12].

While smart decisions while designing ISAs and their extensions have been shown to affect performance, the microarchitecture of the CPU still has a large effect on the performance and energy efficiency of a chip. This is demonstrated by the Forward Slice Core Microarchitecture design, which was simulated to be 64% better than an In-Order core, and a couple dozen percentage points around other microarchitectures [13]. This shows that CPUs with the same ISA can still vary in performance significantly, and this must be controlled for in the experiment. The effect of other parts of the CPU have been shown to also have massive impacts on energy efficiency, especially with register files. Register files are the most frequently used form of memory and are implemented in the silicon of the chip itself. The size of the register file can be, in part, determined by the ISA of the overall CPU, as ISAs that call for more registers in their specifications will naturally result in more registers in the physical register file. A 3.6% increase in the size of a register file has been shown to have a 24% reduction in dynamic energy and 54% reduction in static energy consumption [14]. Part of the reason the register file is so important is that it is used in almost every single operation, due to their crucial role as the most accessible form of memory in the CPU. The register file isn't always needed however, as a 4-level register hierarchy with a bypass network could avoid 25-40% of the register file reads through clever instruction result mapping [15]. This means that instead of storing data directly into the register file after every operation, the results of some operations could bypass the register file entirely and be fed into the next operation. Using the register file less is a great optimization for energy efficiency since a register file access can take 50x the energy of an arithmetic operation [15]. While the register file uses a large amount of energy relative to an arithmetic operation, the register file is estimated to use 10x less energy than an off-chip memory access, meaning decreasing these off-chip memory accesses is also a point of optimization. The issue with just increasing the size of the register file is that, with an energy-aware compiler (encc) and a low-power ARM configuration, an increase from 3 to 8 registers saves 22% of the energy, but there are diminishing returns after the 7-19 register range [16]. While the number of registers in a register file has a significant impact on the energy efficiency of a CPU, the number of ports (inputs and outputs of the register file) can quadratically increase the amount of energy used in a processor. Additionally, split register file microarchitectures have shown promise over traditional, monolithic register files. A split register file configuration can result in a decrease of 9-10% for input data switching activity (selecting which registers to read from),

and a decrease of 12-14% for output data switching activity (selecting which registers to write to). This solution not only reduces the energy consumption for each local register file by 17%, but it also decreases the area required for a register file, increasing processor manufacturing yield and decreasing costs [17].

While microarchitectural improvements can be made, there have been many architectural improvements suggested to increase performance. By accelerating frequently used sequences of instructions, an architecture framework for Transparent Instruction Set Customization was estimated to gain over 2. 2x the average performance of traditional general Configurable Compute Accelerator (CCA) designs [18]. This framework is especially promising since it reduces not only the area (and therefore cost) of the accelerators, but it also reduces the time spent designing and verifying chips. By modifying a general-purpose ISA to interface nicely with this framework, one general purpose processor can be accelerated by any number of accelerator chips, reducing verification costs, decreasing development time, and increasing performance and energy efficiency. Another promising solution is with Composite-ISA cores. A common configuration for a CPU in the embedded space is to have a CPU with multiple ISAs, typically one that targets efficiency and one that targets performance. Composite-ISA cores are different from traditional cross-ISA processors since Composite-ISA cores implement many different versions of the same base ISA across the processor, while a traditional cross-ISA processor implements different ISAs across cores. With Composite-ISA cores, results have shown a performance increase of 19% and an energy-savings of 30% over previous single-ISA heterogeneous designs [19]. These composite-ISA cores can also significantly decrease licensing costs and eliminate the need for cross-ISA binary translation. While RISC ISAs are generally identified in part by their constant-width instructions (meaning each instruction takes up the same amount of bits), an extension to RISC ISAs to give variable-length instructions has been shown to save 15% of data and memory path energy, as well as up to 23% of the execution time of the instruction fetch stage specifically [20].

Previous research has traditionally focused only on a few ISAs, or only measured performance and not efficiency. With this research, I measured both the performance and efficiency of many different ISAs in many different configurations with the most up-to-date specifications possible.

III. Overview

When analyzing the performance of CPUs, it is impractical to test the performance of existing processors due to large amounts of variability in processor design philosophies and goals. Existing RISC processors are typically aimed for energy efficiency, and existing CISC processors are typically aimed for performance. While using existing processors could lead to bias, designing and fabricating laboratory processors is even more impractical due to the high costs and lead times (delays) that would be incurred. Modern researchers, when evaluating the performance of different complexity ISAs, typically either look at a single product line that features evolutions of a single ISA, or the researchers simulate the performance and area of the processor using tools such as the gem5 simulator. gem5 is a discrete-event computer architecture simulator that is useful for testing the designs of different CPU microarchitectures, but can be applied to keep the microarchitectures relatively the same and evaluate the performance of the ISAs instead.

IV. Methodology

Using gem5's modular design, my models were controlled at a very fine level, simulating key hardware components such as caches, memory controllers, buses, I/O Devices, and most importantly, the ISA & type of core (In-Order vs Out-of-Order). Of the many pre-built and industry-standard benchmark suites existing with gem5 out-of-the-box, this study used the GAPBS benchmark suite.

The Graph Algorithm Performance Benchmark Suite (GAPBS) was chosen due to how it heavily tests memory access efficiency across its suite of different graph algorithms, such as for Breadth First Search (BFS), PageRank, Betweenness Centrality, Triangle Counting, and Single-Source Shortest Paths (SSSP). While not indicative of overall performance, the memory-intensiveness of the GAPBS benchmark suite is helpful for indicating which processors are going to be more energy efficient in high memory bandwidth applications. It is important to note that two other benchmark suites that are considered industry standard were not used due to licensing issues, namely SPEC-CPU and PARSEC.

The process for benchmarking each ISA went as follows: a script written using the gem5 simulator framework tested gem5's standard library implementation of base x86, ARM, and RISC-V 64-bit CPUs in configurations consisting of combinations of CPU type and binary type, so the In-Order and Out-of-Order CPU type configurations and the bc, bfs, cc, cc_sv, converter, pr, pr_spmv, sssp, and tc GAPBS benchmarks were tested. All CPUs were tested with the same 4GiB of 4400 MT/s DDR5 RAM with 2 channels and an interleaving size of 64. All CPUs had the amount of cache in an AMD Ryzen 9800x3d when normalizing for each core (32KiB L1 Data Cache, 32KiB L1 Instruction Cache, and 1 MiB of L2 Cache), which simulates a current high-end consumer desktop CPU. The benchmark suite was run in Syscall Emulation mode and not Full System Emulation mode because Full System Emulation (including Linux Kernel bootup) was not needed. All benchmarks were run with a single core to test single-core performance and efficiency.

V. Results

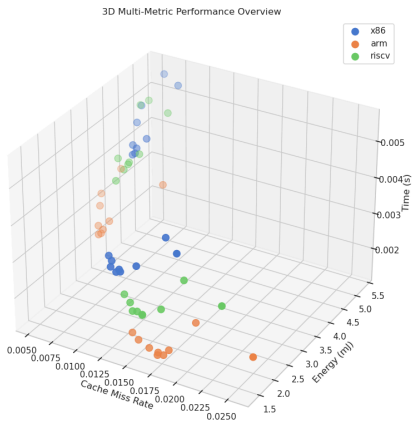

Fig. 1. Time taken for each ISA on each of the benchmarks plotted as a function of Cache Miss Rate and Memory Energy consumed.

In Fig. 1 you can see an overall performance trend of ARM being faster than RISC-V, which is faster than x86. You can also observe the RISC ISAs (ARM and RISC-V) as having higher cache miss rates, which is due to the more complex instruction decoding requirements of x86 when compared to the fixed-length instruction decoders required for ARM and RISC-V in the general-computing sense. The RISC ISAs (ARM and RISC-V) are also observed as having lower energy consumption on average when compared to the CISC ISA (x86).

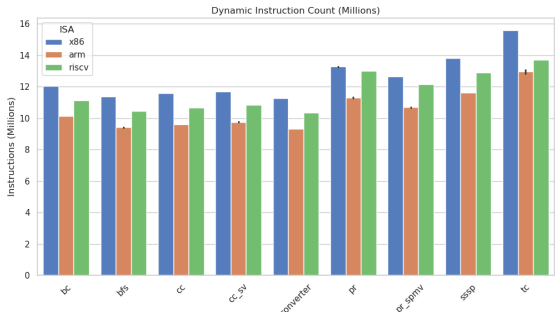

Fig. 2. Amount of instructions in each binary for each ISA in millions of instructions

In Fig. 2, you can see another overall trend with ARM having the lowest dynamic instruction count, RISC-V coming in second, and x86 coming in third. While this may seem counter-intuitive for the CISC instruction set to have the most amount of instructions due to its wide selection of very specialized and niche instructions, I believe that this difference comes down to the difference in compilers (more compiler optimizations typically leads to more instructions, especially-so with x86′s register-memory architecture as compared to ARM and RISC-V's load-store architecture), and possibly due to the effect of register pressure because of x86′s 16 defined registers and ARM and RISC-V's 32 general-purpose registers.

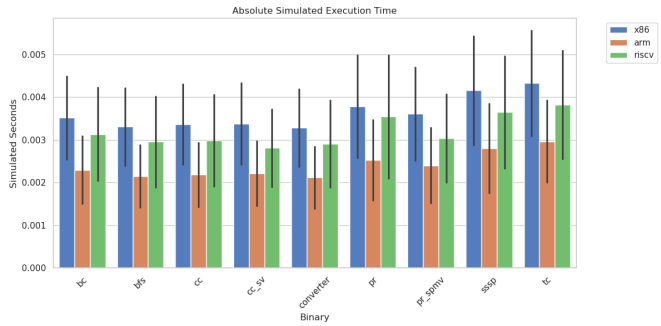

Fig. 3. Amount of time taken for each benchmark and ISA combination

In Fig. 3, we again see that the RISC ISAs lead in performance over the CISC x86 CPU core, possibly because of GAPBS's highly memory-intensive nature.

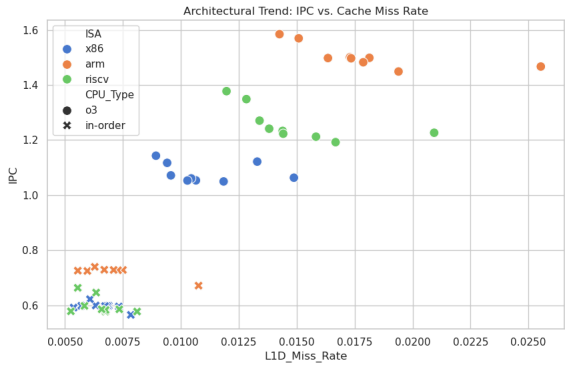

Fig. 4. Number of Instructions Per Cycle (IPC) plotted as a function of the L1 Data Cache (L1D) miss rate

In Fig. 4, ARM stands out as a clear leader in both instructions per clock (IPC) as well as in the L1 Data Cache (L1D) miss rate. This is due to ARM's high reliance on prefetching, which increases performance in the IPC sense, but leads to a less efficient use of memory because of cache thrashing which is caused by an inefficient filling/clearing of cache. Here we can see an trend of high L1D miss rates for RISC ISAs because of their load-store architecture, which requires a larger volume of individual memory access instructions for the same task (1 instruction to load data, x instructions to perform operations on that data, 1 instruction to store that data).

VI. Discussion

VII. Limitations

Since microarchitectures can change a significant amount between CPUs even within the same product line, a large amount of work was done to ensure that results were as comparable as possible using gem5's great level of customization. One limitation of the microarchitectural differences between CPUs is that ISAs can define certain features that make large impacts in the underlying microarchitecture, and it is difficult to pinpoint the exact cause

for overall performance differences due to these effects. For example, RISC-V has fixed-length instructions, making decoding instructions very simple, but x86 has variable-length instructions, making decoding instructions a complex and area-consuming part of the final processor design. Another limitation of the experiment is that the results are also largely dependent on compilers/tooling, meaning that unoptimized or unsupported compiler features can affect the results also. RISC-V is likely to be the ISA with the largest negative (if any) effect due to this, with x86 and ARM having decades of compiler support and corporate backing compared to RISC-V's relatively new compiler collections and industry support. This is also reflected in the ISAs chosen for the benchmarks, as the MIPS, SPARC, and PowerPC ISA compiler collections did not support some of the instructions and syscalls required by the GAPBS Benchmark Suite, making them unsuitable for testing, but this is likely due to their lack of real use in Linux systems in recent years.

References

[1] E. Blem, J. Menon, T. Vijayaraghavan, and K. Sankaralingam, "ISA Wars: Understanding the Relevance of ISA being RISC or CISC to Performance, Power, and Energy on Modern Architectures," *ACM Trans. Comput. Syst.*, vol. 33, no. 1, Mar. 2015, doi: 10.1145/2699682.

[2] G. M. Amdahl, G. A. Blaauw, and F. P. Brooks, "Architecture of the IBM System/360," *IBM Journal of Research and Development*, vol. 8, no. 2, pp. 87–101, Apr. 1964, doi: 10.1147/rd.82.0087.

[3] D. A. Patterson and D. R. Ditzel, "The Case for the Reduced Instruction Set Computer," *ACM SIGARCH Computer Architecture News*, vol. 8, no. 6, pp. 25–33, 1980.

[4] E. Grochowski and M. Annavaram, "Energy per Instruction Trends in Intel® Microprocessors." 2006.

[5] K. M. Dixit, "The SPEC benchmarks," *Parallel Computing*, vol. 17, no. 10–11, pp. 1195–1209, Dec. 1991, doi: 10.1016/s0167-8191(05)80033-x.

[6] J. Lowe-Power and others, "The gem5 Simulator: Version 20.0+," *arXiv preprint arXiv:2007.03152*, 2020, [Online]. Available: https://arxiv.org/abs/2007.03152

[7] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, "McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures," in *2009 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, New York, NY, USA, 2009, pp. 469–480.

[8] N. Slingerland and A. Smith, "Performance Analysis of Instruction Set Architecture Extensions for Multimedia." 2001.

[9] A. Akram and L. Sawalha, "A Study of Performance and Power Consumption Differences Among Different ISAs," in *2019 22nd Euromicro Conference on Digital System Design (DSD)*, Kallithea, Greece, 2019, pp. 628–632. doi: 10.1109/DSD.2019.00098.

[10] D. Suárez, F. Almeida, and V. Blanco, "Comprehensive analysis of energy efficiency and performance of ARM and RISC-V SoCs," *J. Supercomput.*, vol. 80, no. 9, pp. 12771–12789, 2024, doi: 10.1007/s11227-024-05946-9.

[11] A. Akram and L. Sawalha, "The Impact of ISAs on Performance." 2017.

[12] A. Guermouche and A.-C. Orgerie, "Experimental analysis of vectorized instructions impact on energy and power consumption under thermal design power constraints," technical report, 2019.

[13] K. Lakshminarasimhan, A. Naithani, J. Feliu, and L. Eeckhout, "The Forward Slice Core Microarchitecture," in *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques (PACT)*, Sept. 2020, pp. 361–372. doi: 10.1145/3410463.3414629.

[14] M. Khavari Tavana, S. Ahmadian Khameneh, and M. Goudarzi, "Dynamically adaptive register file architecture for energy reduction in embedded processors," *Microprocessors and Microsystems*, vol. 39, no. 2, pp. 49–63, Feb. 2015, doi: 10.1016/j.micpro.2015.01.005.

[15] K. Asanović, "Energy-Exposed Instruction Set Architectures." 2014.

[16] L. Wehmeyer, M. Kumar, S. Steinke, P. Marwedel, and M. Balakrishnan, "Analysis of the Influence of Register File Size on Energy Consumption, Code Size, and Execution Time," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 20, no. 11, pp. 1329–1337, 2001, doi: 10.1109/43.959862.

[17] V. Zyuban and P. M. Kogge, "The energy complexity of register files," in *Proceedings of the 1998 International Symposium on Low Power Electronics and Design (ISLPED)*, Jan. 1998. doi: 10.1145/280756.280943.

[18] N. Clark, J. Blome, M. Chu, S. Mahlke, S. Biles, and K. Flautner, "An Architecture Framework for Transparent Instruction Set Customization in Embedded Processors," in *Proceedings of the 32nd International Symposium on Computer Architecture (ISCA)*, 2005. [Online]. Available: https://cccp.eecs.umich.edu/papers/ntclark-isca05.pdf

[19] A. Venkat, H. Basavaraj, and D. M. Tullsen, "Composite-ISA Cores: Enabling Multi-ISA Heterogeneity Using a Single ISA," in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Washington, DC, USA, 2019, pp. 42–55. doi: 10.1109/HPCA.2019.00026.

[20] M. Collin and M. Brorsson, "Low power instruction fetch using profiled variable length instructions," in *IEEE International [Systems-on-Chip] SOC Conference, 2003. Proceedings.*, Portland, OR, USA, 2003, pp. 183–188. doi: 10.1109/SOC.2003.1241489.